DT059A - Applied Optimization
Lab 3 Report

Kaan Tekin Öztekin[1]

[1]**Department of Computer and Electrical Engineering, Mid Sweden University, Sundsvall, Sweden ,**
Email: {kaoz2500}@student.miun.se ,
Date: [2025-12-21]

1. Introduction

In this lab, numerical optimization algorithms for unconstrained optimization problems were implemented and analyzed. The primary objective was to develop first- and second-order optimization methods from scratch and to study their convergence behavior. In particular, the conjugate gradient method and the modified Newton method were applied to a set of quadratic objective functions. The performance of the algorithms was evaluated in terms of convergence speed, robustness, and sensitivity to problem structure. Graphical visualizations were used to illustrate the optimization paths and to support the numerical results.

2. Experimental Setup

All implementations in this lab were developed using Python. Numerical computations were carried out with the `NumPy` library, while graphical visualizations were generated using `Matplotlib`. No built-in optimization solvers were used; all algorithms were implemented from scratch in accordance with the problem requirements.

Each optimization problem was implemented in a separate script to clearly reflect differences in objective functions, starting points, and algorithmic behavior. For each problem, both the conjugate gradient method and the modified Newton method were applied using identical initial conditions to enable a direct comparison of convergence characteristics.

3. Solutions

3.1. Part A – Conjugate Gradient Method

3.1.1 Problem 10.52

For the implementation of the conjugate gradient algorithm, the objective function is first rewritten in the standard quadratic form

$$f(x) = \frac{1}{2}x^T Hx - b^T x, \tag{1}$$

where H denotes the Hessian matrix and b represents the vector of linear coefficients. For Problem 10.52, these quantities are obtained directly from the analytical expression of the objective function.

Pseudo-code representation:

```
H  <- Hessian matrix of f
b  <- linear term coefficients
x0 <- initial design point
```

$$H = \begin{bmatrix} 2 & -2 \\ -2 & 4 \end{bmatrix}, \qquad b = \begin{bmatrix} 4 \\ 0 \end{bmatrix}, \qquad x^{(0)} = (1,1). \tag{2}$$

These quantities are then used directly in the conjugate gradient algorithm to compute the gradient $\nabla f(x) = Hx - b$ and the exact step size at each iteration. The objective function is defined as

$$f(x_1, x_2) = x_1^2 + 2x_2^2 - 4x_1 - 2x_1x_2, \tag{3}$$

with the starting design point

$$x^{(0)} = (1,1). \tag{4}$$

The function is quadratic and strictly convex since the Hessian matrix H is symmetric positive definite. Therefore, the conjugate gradient method is guaranteed to converge to the global minimum in at most n iterations, where n is the number of design variables. In this case, $n = 2$.

The algorithm was implemented from scratch in Python without using any built-in optimization solvers. At each iteration, the gradient $\nabla f(x)$ and the step size α_k were computed analytically. Using the CG method, convergence was achieved in two iterations, yielding the optimal solution

$$x^* = (4,2), \tag{5}$$

with the corresponding optimal function value

$$f(x^*) = -8. \tag{6}$$

Figure 1 illustrates the contour plot of the objective function together with the conjugate gradient iteration path. The initial design point $x^{(0)}$, the intermediate iterate, and the optimal solution x^* are indicated.

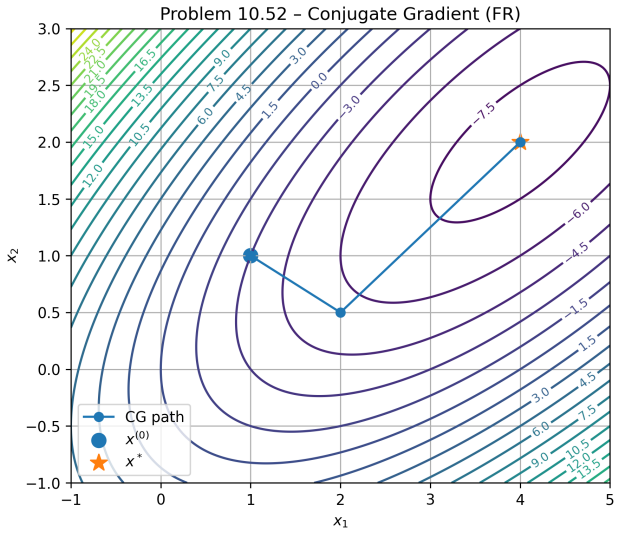

Figure 1. Contour plot of the objective function for Problem 10.52 with the conjugate gradient iteration path. The algorithm starts from $x^{(0)} = (1,1)$ and converges to the optimal solution $x^* = (4,2)$ in two iterations.

It is noted that the reference course book reports the intermediate iterate $x^{(2)} = \left(\frac{5}{2}, \frac{3}{2}\right)$ for this problem, which corresponds to the *steepest descent* method with exact line search. In contrast, the present assignment requires the application of the conjugate gradient method.

3.1.2 Problem 10.54

The unconstrained quadratic objective function is given by

$$f(x_1, x_2) = 6.983\, x_1^2 + 12.415\, x_2^2 - x_1, \tag{7}$$

with the initial design point

$$x^{(0)} = (2,1). \tag{8}$$

The function is quadratic and strictly convex, since its Hessian matrix

$$H = \begin{bmatrix} 13.966 & 0 \\ 0 & 24.83 \end{bmatrix} \tag{9}$$

is symmetric positive definite. Consequently, the conjugate gradient method with exact line search is guaranteed to converge to the global minimum in at most $n = 2$ iterations.

Starting from $x^{(0)}$, the method converged in two iterations to the optimal solution

$$x^* = \left(\frac{1}{13.966}, 0\right) \approx (0.0716,\ 0), \tag{10}$$

with the corresponding optimal function value

$$f(x^*) \approx -0.0358. \tag{11}$$

Figure 2 presents the contour plot of the objective function together with the conjugate gradient iteration path. Due to the diagonal structure of the Hessian matrix, the level sets are axis-aligned ellipses. As observed in the figure, the algorithm reaches the optimal point in only two iterations, demonstrating the strong convergence properties of the conjugate gradient method for quadratic objective functions.

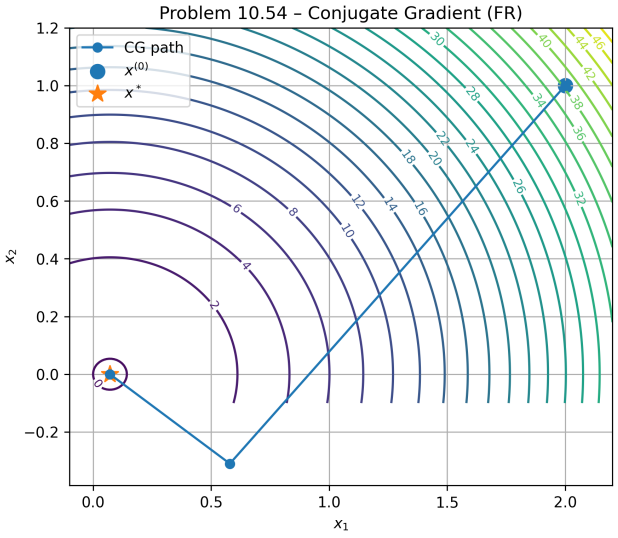

Figure 2. Contour plot of the objective function for Problem 10.54 with the conjugate gradient iteration path. The algorithm starts from $x^{(0)} = (2, 1)$ and converges to the optimal solution x^* in two iterations.

It is noted that the reference course book reports the intermediate iterate $x^{(2)} \approx (0.222,\ 0.0778)$ for this problem, which corresponds to the steepest descent method with exact line search.

3.1.3 Problem 10.56

In this problem, the unconstrained quadratic objective function is defined as

$$f(x_1, x_2) = 25x_1^2 + 20x_2^2 - 2x_1 - x_2, \tag{12}$$

with the initial design point

$$x^{(0)} = (3, 1). \tag{13}$$

The function is quadratic and strictly convex, since its Hessian matrix

$$H = \begin{bmatrix} 50 & 0 \\ 0 & 40 \end{bmatrix} \tag{14}$$

is symmetric positive definite. Consequently, the conjugate gradient method with exact line search is guaranteed to converge to the global minimum in at most $n = 2$ iterations. Starting from $x^{(0)}$, the method converged in two iterations to the optimal solution

$$x^* = \left(\frac{2}{50}, \frac{1}{40}\right) = (0.04,\ 0.025), \tag{15}$$

with the corresponding optimal function value

$$f(x^*) = -0.0525. \tag{16}$$

Figure 3 shows the contour plot of the objective function together with the conjugate gradient iteration path. Due to the strong anisotropy of the quadratic surface, the level sets form highly elongated ellipses, indicating a significant difference in curvature between the x_1 and x_2 directions. As illustrated in the figure, the algorithm rapidly moves toward the minimum and converges in only two iterations, highlighting the robustness of the conjugate gradient method for quadratic problems with varying curvature.

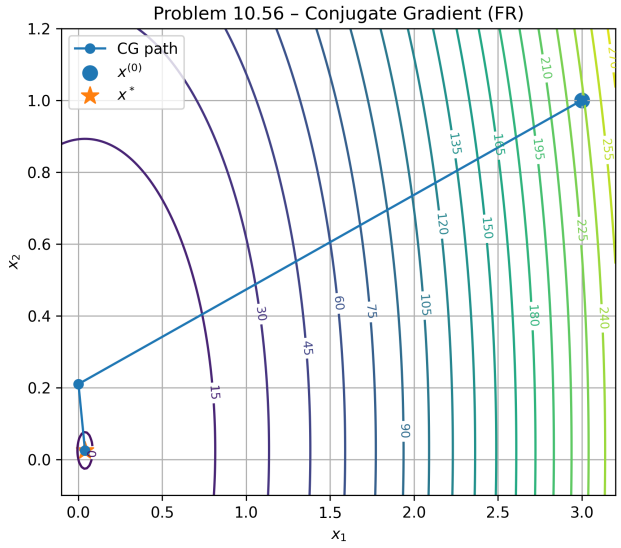

Figure 3. Contour plot of the objective function for Problem 10.56 with the conjugate gradient iteration path. The algorithm starts from $x^{(0)} = (3, 1)$ and converges to the optimal solution x^* in two iterations.

It is noted that the reference course book reports the intermediate iterate $x^{(2)} \approx (0.0490,\ 0.0280)$ for this problem, which corresponds to the steepest descent method with exact line search.

3.2. Part B – Modified Newton's Method

The modified Newton method is a second-order iterative optimization algorithm designed for unconstrained minimization problems. The modified Newton approach explicitly enforces the descent condition at each iteration to ensure robustness, particularly when the Hessian matrix is not positive definite.

Starting from an initial design point $x^{(0)}$, the algorithm proceeds as follows:

1. Select an initial design vector $x^{(0)}$, set the iteration counter $k = 0$, and choose a tolerance ε for the stopping criterion.
2. Compute the gradient vector
$$c^{(k)} = \nabla f\left(x^{(k)}\right). \tag{17}$$
If $\|c^{(k)}\| < \varepsilon$, terminate the algorithm.
3. Compute the Hessian matrix
$$H^{(k)} = \nabla^2 f\left(x^{(k)}\right). \tag{18}$$
4. Determine the search direction $d^{(k)}$ by solving the linear system
$$H^{(k)} d^{(k)} = -c^{(k)}. \tag{19}$$
If the descent condition
$$c^{(k)T} d^{(k)} < 0 \tag{20}$$

is not satisfied, the Hessian matrix is modified by adding a positive scalar multiple of the identity matrix until a descent direction is obtained.

5. Compute the step size α_k using a one-dimensional search method that minimizes $f\left(x^{(k)} + \alpha d^{(k)}\right)$.
6. Update the design vector

$$x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}. \tag{21}$$

7. Set $k = k + 1$ and return to Step 2.

This modification guarantees that the computed search direction is a descent direction, thereby improving the robustness of the Newton method for general nonlinear optimization problems.

3.2.1 Problem 10.52

Since the objective function is quadratic and its Hessian matrix is symmetric positive definite, the Newton direction satisfies the descent condition without requiring modification.

Figure 4 illustrates the contour plot of the objective function together with the modified Newton iteration path. The algorithm converges rapidly to the optimal solution

$$x^* = (4, 2), \tag{22}$$

with the corresponding minimum function value

$$f(x^*) = -8. \tag{23}$$

As observed in the figure, the modified Newton method reaches the optimum in a single effective step, demonstrating the fast convergence properties of second-order methods for quadratic objective functions.

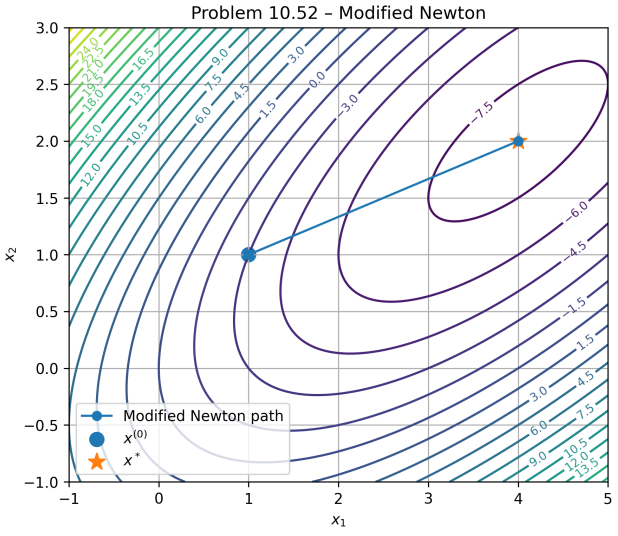

Figure 4. Contour plot and modified Newton iteration path for Problem 10.52.

3.2.2 Problem 10.54

For Problem 10.54, the modified Newton method was initialized at $x^{(0)} = (2, 1)$. Due to the diagonal and positive definite Hessian matrix, the Newton direction is immediately a descent direction.

Figure 5 shows the contour plot and the modified Newton iteration path. The method converges in a single iteration to the optimal solution

$$x^* = \left(\frac{1}{13.966}, 0\right), \tag{24}$$

with a corresponding minimum function value of approximately $f(x^*) = -0.0358$.

The straight-line path observed in the figure reflects the fact that the quadratic model is solved exactly by the Newton update.

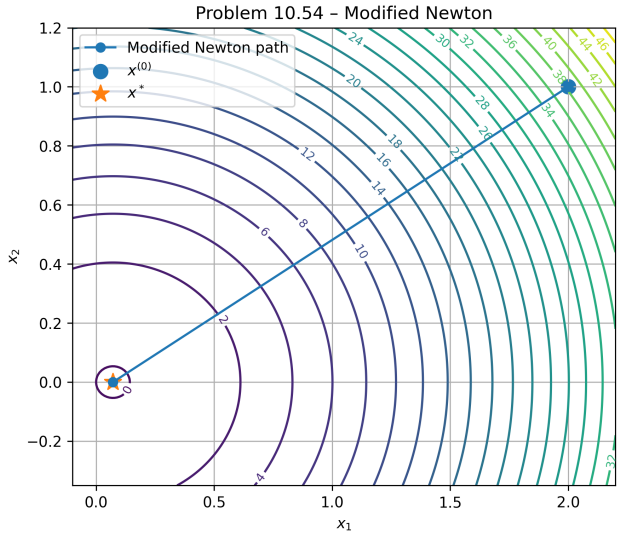

Figure 5. Contour plot and modified Newton iteration path for Problem 10.54.

3.2.3 Problem 10.56

The modified Newton method was applied to Problem 10.56 starting from $x^{(0)} = (3, 1)$. Despite the strong anisotropy of the quadratic surface, the Hessian matrix remains positive definite, allowing the Newton direction to satisfy the descent condition.

Figure 6 illustrates the rapid convergence behavior of the method. The algorithm converges directly to the optimal solution

$$x^* = (0.04,\ 0.025), \tag{25}$$

with the corresponding minimum function value

$$f(x^*) = -0.0525. \tag{26}$$

The figure highlights the ability of the modified Newton method to efficiently handle problems with significantly different curvature scales.

3.2.4 Comparison of Convergence Speed and Robustness

For the quadratic problems considered in this study, both the conjugate gradient and the modified Newton methods converge rapidly to the global minimum. The conjugate gradient method converges in at most n iterations for strictly convex quadratic functions, which was observed for all test cases. The modified Newton method converges even faster, reaching the optimum in a single effective iteration due to the availability of exact second-order information.

In terms of robustness, the conjugate gradient method relies only on first-order information and does not require the Hessian matrix, making it more reliable for large-scale or poorly conditioned problems. The modified Newton method, while faster for well-conditioned quadratic problems, requires additional safeguards to ensure a descent direction when the Hessian matrix is not positive definite. Therefore, the conjugate gradient method is generally more robust, whereas the modified Newton method offers superior convergence speed when second-order information is reliable.

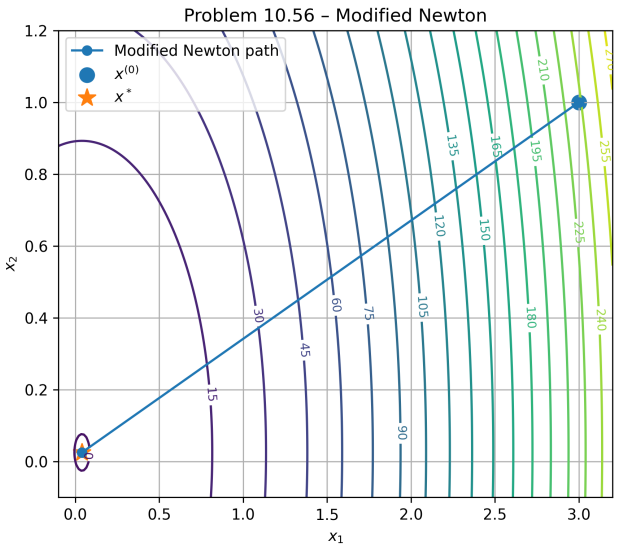

Figure 6. Contour plot and modified Newton iteration path for Problem 10.56.

4. Appendix: Code Snippets

All implementations are done locally in Python using PyCharm and are available on GitHub: https://github.com/kaanoztekin99/applied-optimization/tree/main/lab3